

Communication Models

Topics, Services, Actions, and Parameters

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 3, Lecture A



Lecture Overview

Roadmap

Last week everything was a topic. Today: the **four** ways ROS 2 nodes communicate — **topics**, **services**, **actions**, **parameters** — each defined in turn, how to tell which you need, and how to write the contract down so a compiler catches the mistake, not a robot at 3 a.m.

- 1 Interaction Models
- 2 Interface Definitions
- 3 Topics
- 4 Services
- 5 Actions
- 6 Parameters
- 7 Application to a Mobile Robot

Part 1

Interaction Models

- 1 Interaction Models
- 2 Interface Definitions
- 3 Topics
- 4 Services
- 5 Actions
- 6 Parameters
- 7 Application to a Mobile Robot

Selecting an Interaction Model

“Is this a **stream**, a **question**, or a **job**?”

Stream

never ends
nobody waits

topic

Question

answered at once
caller waits

service

Job

takes time
may be cancelled

action

Getting this wrong does not produce an error message. It produces a robot that works in the lab and hangs on demo day.

If the caller would be unhappy blocking until it finishes, it is not a service. If nobody is waiting for an answer at all, it is a topic.

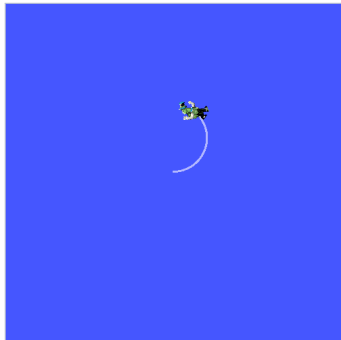
Three Ways to Choose Wrongly

What you wrote	What you wanted	What happens
"Reset the odometry" on a topic	a service	Fire and forget. Did anyone receive it? Did it succeed? Was anyone listening? You do not know, and nothing tells you.
"Navigate to the kitchen" as a service	an action	Blocked for forty seconds. No feedback, no cancellation, no visibility. An obstacle appears and there is no way to say so.
A service called syn- chronously inside a call- back	<code>call_async</code> + a future	<code>spin</code> runs one callback at a time. Yours waits for a reply <code>spin</code> must deliver — so it waits for itself.

In the field

Row 1 is why `/cmd_vel` is a topic and "reset odometry" is not: a dropped velocity is corrected 50 ms later, a dropped reset is silently never done. Row 3 you cause on purpose in session B — easier to remember once you have watched your own node hang.

Turtlesim: One Node, All Four Mechanisms



```
ros2 run turtlesim turtlesim_node
```

Every mechanism in this lecture is already running in that window:

a stream	/turtle1/pose
a question	/spawn
a job	/turtle1/rotate_absolute
a setting	background_r

We will not describe the four mechanisms. We will interrogate a node that has them.

Node Introspection: `ros2 node info`

```
$ ros2 node info /turtlesim
/turtlesim
Subscribers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
Publishers:
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /rosout: rcl_interfaces/msg/Log
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
Service Servers:
  /clear: std_srvs/srv/Empty
  /kill: turtlesim/srv/Kill
  /reset: std_srvs/srv/Empty
  /spawn: turtlesim/srv/Spawn
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  /turtle1/teleport_relative: turtlesim/srv/TeleportRelative
  /turtlesim/describe_parameters: rcl_interfaces/srv/DescribeParameters
  /turtlesim/get_parameter_types: rcl_interfaces/srv/GetParameterTypes
  /turtlesim/get_parameters: rcl_interfaces/srv/GetParameters
  /turtlesim/get_type_description: type_description_interfaces/srv/GetTypeDescription
  /turtlesim/list_parameters: rcl_interfaces/srv/ListParameters
  /turtlesim/set_parameters: rcl_interfaces/srv/SetParameters
  /turtlesim/set_parameters_atomically: rcl_interfaces/srv/SetParametersAtomically
Service Clients:

Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:
```

Four headings, four mechanisms. **Publishers/Subscribers** are topics, **Service Servers** services, **Action Servers** actions, and the `*_parameters` services are how parameters are reached. The first command to run on a node you did not write.

Part 2

Interface Definitions

- 1 Interaction Models
- 2 Interface Definitions**
- 3 Topics
- 4 Services
- 5 Actions
- 6 Parameters
- 7 Application to a Mobile Robot

Message Definition Syntax

A `.msg` file is a list of *type name* pairs. That is all it is.

`my_robot_interfaces/msg/PowerStatus.msg`

```
std_msgs/Header  header      <- stamp + frame_id
float64          voltage     <- pack terminal voltage, volts
float64          percentage   <- 0.0 empty, 1.0 full
bool             charging
```

Primitives	<code>bool int8 int32 int64 float32 float64 string</code>
Arrays	<code>float64[] unbounded, float64[16] fixed, float64[<=8] bounded</code>
Nested	<code>any other message type, e.g. geometry_msgs/Pose</code>

These compile into Python and C++ classes. A misspelt field is a build error.

Standard Message Definition: LaserScan

```
# will make fairly laser-specific assumptions about this data

std_msgs/Header header # timestamp in the header is the acquisition time of
    builtin_interfaces/Time stamp
        int32 sec
        uint32 nanosec
    string frame_id
        # the first ray in the scan.
        #
        # in frame frame_id, angles are measured around
        # the positive Z axis (counterclockwise, if Z is up)
        # with zero angle being forward along the x axis

float32 angle_min        # start angle of the scan [rad]
float32 angle_max        # end angle of the scan [rad]
float32 angle_increment   # angular distance between measurements [rad]

float32 time_increment    # time between measurements [seconds] - if your scanner
                          # is moving, this will be used in interpolating position
                          # of 3d points
float32 scan_time        # time between scans [seconds]

float32 range_min        # minimum range value [m]
float32 range_max        # maximum range value [m]

float32[] ranges          # range data [m]
                          # (Note: values < range_min or > range_max should be discarded)
float32[] intensities     # intensity data [device-specific units]. If your
                          # device does not provide intensities, please leave
                          # the array empty.
```

Run this before writing a line against an unfamiliar topic. Never out of date, and exactly the field list your code must use. `ros2 topic type /scan` names the one a topic carries.

The Role of the Message Header

```
$ ros2 topic echo /scan --once --field header
stamp:
  sec: 524
  nanosec: 800000000
frame_id: base_scan
---
$ ros2 topic echo /scan --once --field angle_increment
0.01749303564429283
---
$ ros2 topic echo /scan --once --field range_min
0.11999999731779099
---
$ ros2 topic echo /scan --once --field range_max
3.5
---
```

stamp

When the measurement was taken — not when it arrived. Two nodes disagree about the world by exactly that difference.

frame_id

Which coordinate frame the numbers are in. A range of 2.4 m is meaningless until you know from where.

A real /scan from the TurtleBot 3, `-field` printing one field instead of nine hundred ranges. `frame_id: base_scan` — the message says *where* it was measured, not just *what*; Week 7 is about that line. A stamped, framed message can be replayed, fused with another sensor and transformed into any frame. An unstamped one can only be printed.

If it came from the physical world, it needs a stamp and a frame.

Count the Separators: Service versus Action

```
$ ros2 interface show turtlesim/srv/Spawn
float32 x
float32 y
float32 theta
string name # Optional. A unique name will be created and returned if this is empty
---
string name
```

srv/Spawn: one ---

```
$ ros2 interface show turtlesim/action/RotateAbsolute
# The desired heading in radians
float32 theta
---
# The angular displacement in radians to the starting position
float32 delta
---
# The remaining rotation in radians
float32 remaining
```

action/RotateAbsolute: two

One separator is a service — request above the line, response below: send `x`, `y`, `theta`, `name`, get the name back. **Two** is an action — goal, result, feedback: which is what an action *is*. Something you ask for, something you eventually get, and something you hear about while you wait.

Custom Definitions: When to Write One, and Where It Lives

Define one when	Do <i>not</i> when
The data genuinely has no standard type	A standard type already fits — use it
Fields must travel together, atomically, with one timestamp	You only want to rename fields
It is your own robot's internal state	You are avoiding reading the standard definition

Definitions live in their own `ament_cmake` package (`<project>_interfaces`, holding *only* `.msg/.srv/.action` files); nodes stay in an `ament_python` one (`<project>_nodes`). A Python package **cannot** generate interfaces — that is a CMake job — and whoever wants to *talk* to your robot needs the definitions, not your code. It is why `sensor_msgs` is a package of its own.

Every custom type is a wall. Publish `MyLaserScan` and `RViz2` cannot display it.

Part 3 Topics

- 1 Interaction Models
- 2 Interface Definitions
- 3 Topics**
- 4 Services
- 5 Actions
- 6 Parameters
- 7 Application to a Mobile Robot

Definition: Topic

Topic

A **topic** is a named, typed, unidirectional channel. Publishers send messages to the name; every subscriber connected to that name receives every message sent while it is connected. Neither end knows the other exists, and neither end is told whether the other is there.

Pattern	Publish / subscribe. Many-to-many: any number of publishers, any number of subscribers, all anonymous.
Synchrony	Asynchronous. The publisher does not block and receives no acknowledgement. There is no reply of any kind.
Definition file	.msg — one message type, one list of fields.
Inspect with	ros2 topic list, echo, info, hz

Use a topic when data flows continuously and nobody is waiting for an answer.

Topic Inspection, and What It Does Not Tell You

```
ros2 topic echo /turtle1/pose --once
x: 6.175500869750977
y: 7.55879020690918
theta: 2.5055999755859375
linear_velocity: 0.0
angular_velocity: 0.0
---
```

-once prints one sample and exits. Without it this scrolls forever — which is the point: the turtle publishes whether or not anyone listens.

Property	What it costs you
A publisher is not told who subscribes	Publishing into an empty topic is not an error. It looks identical to working.
A publisher is not told the message arrived	There is no delivery report. <code>ros2 topic hz</code> on the <i>subscriber</i> side is how you find out.
Subscribers join and leave at any time	The first few messages after startup are routinely lost while discovery completes.
Both ends must agree on the type, <i>and</i> on the QoS	Neither mismatch is a runtime error — the connection is simply never made. QoS is at the end of the lecture.

Every failure mode of a topic looks the same from the publisher: nothing happens.

Part 4 Services

- 1 Interaction Models
- 2 Interface Definitions
- 3 Topics
- 4 Services**
- 5 Actions
- 6 Parameters
- 7 Application to a Mobile Robot

Definition: Service

Service

A **service** is a named, typed request/reply call. A client sends one request to exactly one server, which does the work and returns exactly one response. With no server running under that name there is nobody to answer.

Pattern	Request / reply. One client to one named server — and the client knows it is talking to somebody.
Synchrony	One request, one response, no progress in between. In <code>rcldpy</code> the client is <i>always</i> asynchronous: <code>call_async</code> returns a future.
Definition file	<code>.srv</code> — request, ---, response.
Inspect with	<code>ros2 service list</code> , <code>type</code> , <code>call</code>

Use a service when you need an answer, and the work is short enough that waiting is reasonable.

Asynchronous Service Clients

```
self.cli = self.create_client(Spawn, 'spawn')
req = Spawn.Request()
req.x, req.y, req.name = 2.0, 8.0, 'scout'

future = self.cli.call_async(req)
future.add_done_callback(self.on_reply)

def on_reply(self, future):
    self.get_logger().info(f'spawned: {future.result().name}')
```

`call_async` returns immediately with a **future** — a promise of a result later. You attach a callback and return control to `spin`, which is then free to deliver the reply.

This shape avoids failure 3. Get used to it now; it returns for actions.

A Service from the Terminal: Find It, Then Call It

```
$ ros2 service list -t | grep -v parameter
/clear [std_srvs/srv/Empty]
/kill [turtlesim/srv/Kill]
/reset [std_srvs/srv/Empty]
/robot_state_publisher/get_type_description [type_description_interfaces/srv/GetTypeDescription]
/ros_gz_bridge/get_type_description [type_description_interfaces/srv/GetTypeDescription]
/spawn [turtlesim/srv/Spawn]
/turtle1/set_pen [turtlesim/srv/SetPen]
/turtle1/teleport_absolute [turtlesim/srv/TeleportAbsolute]
/turtle1/teleport_relative [turtlesim/srv/TeleportRelative]
/turtlesim/get_type_description [type_description_interfaces/srv/GetTypeDescription]
```

`list -t`: name and type

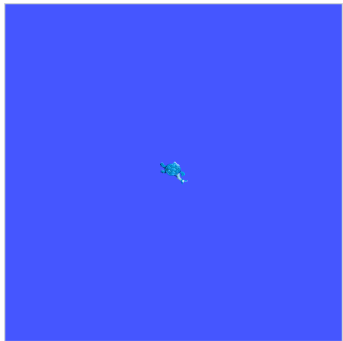
```
$ ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 8.0, theta: 0.0, name: \"scout\"}"
waiting for service to become available...
requester: making request: turtlesim.srv.Spawn_Request(x=2.0, y=8.0, theta=0.0, name='scout')

response:
turtlesim.srv.Spawn_Response(name='scout')
```

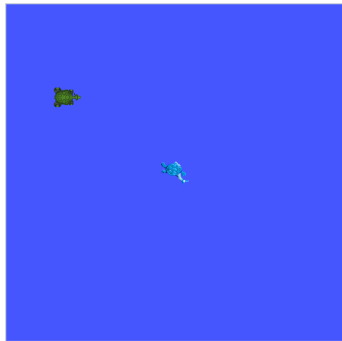
`call`: request, then response

The six `*_parameters` entries are the ones every node gets free. You can exercise a server completely from the terminal *before its client exists* — so you test the two halves separately and know which one is broken.

Observed Effect of a Service Call



before the call — one turtle



after the call — scout, at (2,8)

A service call changed the world and told you it had. A topic would have done neither.

Part 5 Actions

- 1 Interaction Models
- 2 Interface Definitions
- 3 Topics
- 4 Services
- 5 Actions**
- 6 Parameters
- 7 Application to a Mobile Robot

Definition: Action

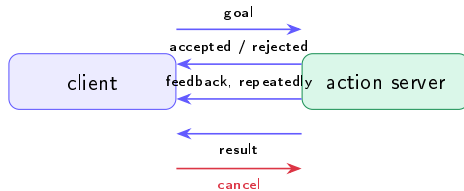
Action

An **action** is a named, typed transaction for work that takes time. A client sends a goal; the server accepts or rejects it, publishes feedback while it runs, and returns one result. Either side may cancel. Underneath it is three topics and two services, so everything you know about both already applies.

Pattern	Goal / feedback / result, plus cancel. One client, one named server, over a long interaction rather than a single exchange.
Synchrony	Asynchronous throughout. The client is never blocked; progress arrives as feedback, and the result arrives once.
Definition file	<code>.action — goal, ---, result, ---, feedback.</code>
Inspect with	<code>ros2 action list, info, send_goal --feedback</code>

Use an action when the caller needs progress, or the right to change its mind.

The Action Protocol



Goal accepted	With an ID. The server could have rejected it.
Feedback	Repeatedly — remaining counting to zero while the turtle turns. A service would have shown you nothing here.
Result	Once — delta, the displacement back to where it started.
Status	SUCCEEDED. Not every goal ends this way; that is why it exists.

Accepted → feedback → result → status. Every action, Nav2's included, is this.

Action Feedback During Execution

```
$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 0.12}" --feedback
Waiting for an action server to become available...
Sending goal:
  theta: 0.12

Goal accepted with ID: b3d60a2cfff04b158858822b3edc49a3

Feedback:
  remaining: 0.10399999469518661

Feedback:
  remaining: 0.08799999952316284

Feedback:
  remaining: 0.07199999690055847

Feedback:
  remaining: 0.0559999942779541

Feedback:
  remaining: 0.03999999910593033

Feedback:
  remaining: 0.023999996483325958

Feedback:
  remaining: 0.007999993860721588

Result:
  delta: -0.09600000083446503

Goal finished with status: SUCCEEDED
```

--feedback is the flag that shows what an action gives you and a service cannot: **you can watch it work.**

Part 6 Parameters

- 1 Interaction Models
- 2 Interface Definitions
- 3 Topics
- 4 Services
- 5 Actions
- 6 Parameters**
- 7 Application to a Mobile Robot

Definition: Parameter

Parameter

A **parameter** is a named, typed value belonging to one node, declared with a default at startup and readable and writable from outside the process while the node runs. It is the only one of the four that is a property of a node rather than a conversation between nodes.

Pattern	Key / value, per node. Reached over services every node creates for itself, unasked.
Synchrony	A set takes effect at once — no restart, no rebuild. The node sees the new value the next time it reads it.
Definition file	None to write — declared in code with <code>declare_parameter</code> ; saved and restored as YAML.
Inspect with	<code>ros2 param list</code> , <code>get</code> , <code>set</code> , <code>dump</code>

Use a parameter for a number you will want to change without editing and rebuilding code.

Declaring Parameters

```
self.declare_parameter('max_speed', 0.22)
speed = self.get_parameter('max_speed').value
```

Declared once with a default, then read where needed. Overridden from outside without touching the code — at launch with `--ros-args -p max_speed:=0.15`, or, for anything reproducible, in a launch file with `parameters=[{'max_speed': 0.15}]`.

Why this is not over-engineering

Every tuning constant in this course — gains in Week 5, the obstacle threshold in Week 6, costmap sizes in Week 11 — is a parameter. If tuning needs an edit and a rebuild you will try five values. If it is a parameter you will try fifty.

Parameters at Runtime: List Them, Then Change One

```
background_b
background_g
background_r
holonomic
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
qos_overrides./parameter_events.publisher.history
qos_overrides./parameter_events.publisher.reliability
start_type.description_service
use_sim_time
```

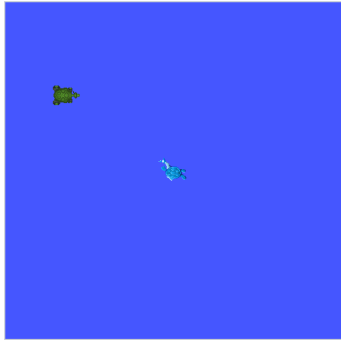
list: four declared

```
$ ros2 param get /turtlesim background_r
Integer value is: 69
$ ros2 param set /turtlesim background_r 200
Set parameter successful
$ ros2 param set /turtlesim background_g 60
Set parameter successful
$ ros2 param set /turtlesim background_b 60
Set parameter successful
```

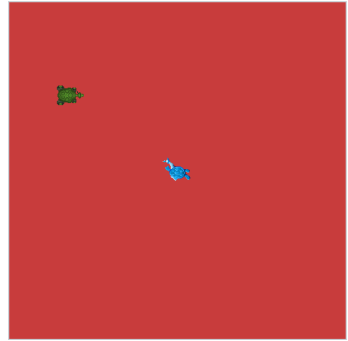
get, set, get

use_sim_time and the qos_overrides come from rcplpy itself, on every node you will ever write. No restart, no rebuild, no edit: Set parameter successful is the node confirming it took the value.

Observed Effect of a Parameter Change



`r,g,b = 69,86,255` — the default



`r,g,b = 200,60,60` — three param set calls later

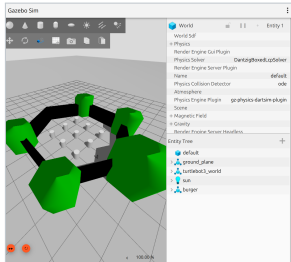
A parameter is not configuration you read at startup. It is state you can reach into.

Part 7

Application to a Mobile Robot

- 1 Interaction Models
- 2 Interface Definitions
- 3 Topics
- 4 Services
- 5 Actions
- 6 Parameters
- 7 Application to a Mobile Robot**

The Same Mechanisms on a Mobile Robot



Shape	Turtlesim	TurtleBot 3
stream	/turtle1/pose	/scan, /odom
stream (in)	/turtle1/cmd_vel	/cmd_vel
question	/spawn	/reset_simulation
job	/turtle1/ rotate_absolute	/navigate_to_pose (Week 11)

TurtleBot 3 Burger in `turtlebot3_world`; from Week 4 every lab runs here. The entity tree is the simulator's — the ROS graph is a separate thing entirely.

The mechanisms are the framework's, not the robot's. That is the point of learning them on a turtle.

What a Topic Costs, and What It Promises

```
$ timeout 8 ros2 topic hz /scan
WARNING: topic [/scan] does not appear to be published yet
average rate: 4.993
min: 0.195s max: 0.206s std dev: 0.00380s window: 7
average rate: 4.992
min: 0.195s max: 0.206s std dev: 0.00332s window: 12
average rate: 4.992
min: 0.194s max: 0.206s std dev: 0.00374s window: 17
average rate: 5.001
min: 0.194s max: 0.206s std dev: 0.00371s window: 23
```

hz: five a second, forever

```
$ ros2 topic info /scan --verbose
Type: sensor_msgs/msg/LaserScan
Publisher count: 1
Node name: ros_gz_bridge
Node namespace: /
Topic type: sensor_msgs/msg/LaserScan
Topic type hash: RIHS01 64c191398013af96509d518dac71d5164f9382553fce5c1f8cca5be7924bd828
Endpoint type: PUBLISHER
GID: 01.0f.46.da.63.6f.94.54.00.00.00.00.00.00.1a.03
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
Subscription count: 0
```

info --verbose: reliability, durability, history

Five a second whether anyone subscribes or not — now imagine calling a service that often and waiting for each reply. **Both ends must agree on that QoS profile, and a mismatch fails silently:** no error, no warning, echo simply prints nothing.

Comparison of the Four Mechanisms

	Topic	Service	Action	Parameter
Shape	stream	question	job	setting
Between	node and node	node and node	node and node	you and one node
Direction	one way	round trip	round trip + updates	read / write
Waiting	nobody waits	caller waits	caller carries on	returns at once
Cancel	n/a	no	yes	n/a
Progress	n/a	no	yes	n/a
Typical	/scan, /cmd_vel	reset, query	navigate, follow	max_speed
Cost	cheapest	cheap	most machinery	cheap

Choose the cheapest shape that answers your question. Most robot traffic is topics, and that is correct.

Session B Outline and Deliverables

- 1 Build an interfaces package — one custom message, one service.
- 2 Publish and subscribe with your own type.
- 3 Write a service server, call it from the terminal, then from a client.
- 4 **Cause the deadlock on purpose.** Then fix it with `call_async`.
- 5 Add parameters, and change one while the node runs.
- 6 Send a goal to a real action server and watch the feedback.

Assignment 1 is due at the end of this week.

Next week the abstraction ends: `/cmd_vel` reaches a real differential drive, and the robot moves.